Cache Sizing: Working Set, the 80/20 Rule & Redis Nodes

how much memory, and how many nodes

Watch the 3-min video

In one line: estimate the working set (data actually read in a period), cache its hot fraction (80/20), multiply by entry size plus overhead, then add headroom and replicas to get a node count, and check throughput per node.

1 Too small or too large

Your URL shortener serves **1 billion redirects a day**, and you want Redis in front of the database. How much memory should the cache have? 10 GB? A terabyte? Guess too small and the **hit rate collapses**, sending traffic back to the database. Guess too large and you **pay for memory nobody uses**.

2 Start with the working set

The **working set** is the data actually accessed during a period, such as a day. It is usually far smaller than the total data. Years of links exist, but most redirects hit links created recently, or a few viral ones. You only need to cache the working set, not everything.

Then apply the **80/20 rule**: about 20% of items get 80% of the traffic. Real traffic is often even more skewed. So caching the hottest 20% of what is read each day usually gives a high hit rate. Measure the real distribution when you can.

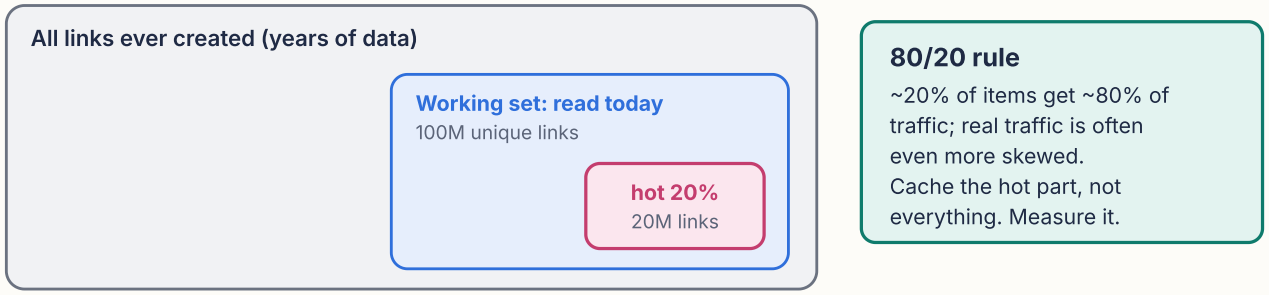


Figure 1. Total data ⊃ working set (read today) ⊃ hot 20%. The cache only needs the inner boxes.

3 Size per entry: value plus overhead

The value is only part of the cost. A short code plus a long URL might be about **500 bytes**. Redis adds per-key overhead: pointers, expiry data and allocator fragmentation, often **50–100 bytes per key**, sometimes more. For small entries, a safe estimate is to **add about 50%**.

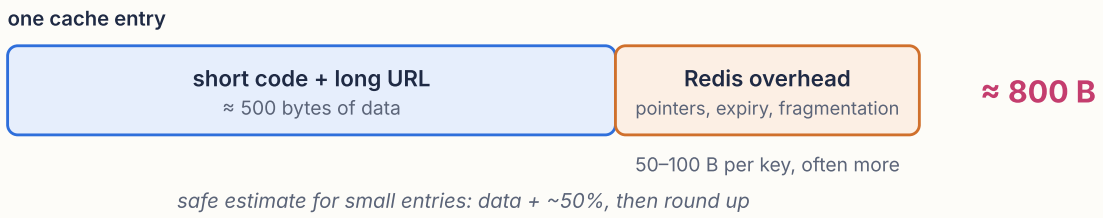


Figure 2. 500 bytes of data becomes about 750 B with overhead; round up to 800 B for easy maths.

4 Worked example

WORKED EXAMPLE: URL SHORTENER CACHE

Working set: 1 billion redirects a day, but say only **100 million unique links** are read.

Hot fraction: cache 20% = **20 million entries**.

Size: 500 B + overhead, round to 800 B. 20M × 800 B = **16 GB**.

Upper bound: if we had sized for 20% of *all* 1 billion reads (200M entries), that is 200M × 800 B = **160 GB**. Still very manageable, so size for 160 GB to be safe.

5 From memory to nodes

Don't fill memory to the top. Leave about **25% headroom** for spikes, fragmentation and snapshots (Redis forks to write RDB snapshots, and pages changed during the fork are copied). A 64 GB node gives about 48 GB usable. 160 GB ÷ 48 GB = 3.3, so **4 primaries**. Add one replica each for availability: **8 nodes in total**.

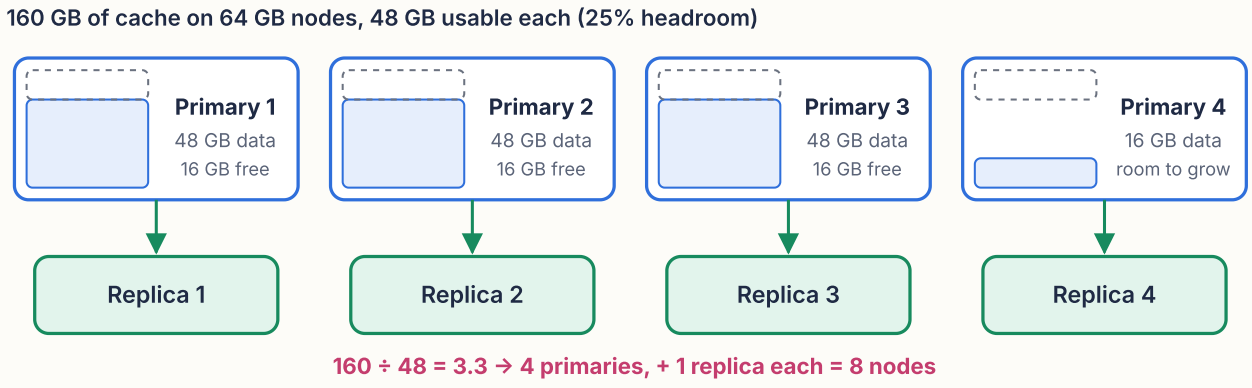


Figure 3. Four primaries hold the 160 GB with headroom; each has a replica that takes over if it fails.

Throughput matters too. One Redis node handles around **100,000 operations per second**. 1 billion redirects a day is about 11,600 per second on average; even at a 3× peak (≈35,000/s) four primaries are far from their limit. If peak ops were 1 million per second, you would need at least 10 primaries for throughput, whatever the memory says.

6 Validate with the hit rate

Track the **hit rate** in production. If it drops, either the working set grew or your eviction policy is wrong. Use an eviction policy such as **least recently used** (Redis `maxmemory-policy allkeys-lru`) and **time-to-live** values, so cold data leaves the cache automatically.

COMMON MISTAKES

- Sizing the cache for the whole dataset instead of the working set.
- Counting only the value bytes and ignoring per-key overhead.
- Filling nodes to 100%: no room for spikes, fragmentation or snapshots.
- Checking memory but not operations per second per node.
- No eviction policy or TTLs, so the cache fills with cold data.

7 Key terms

Working set: data actively accessed in a time window

Per-key overhead: metadata Redis keeps per key

Eviction policy: what to drop when full, e.g. LRU

80/20 rule: ~20% of items get ~80% of traffic

Fragmentation: memory lost between allocations

TTL: time-to-live; key expires

Hit rate: share of reads served from cache

Headroom: spare memory, ~25%

Replica: copy of a primary for failover

8 Interview questions

Q1. What is a working set? BASIC

The subset of data actively accessed over a time window, such as a day. It is usually far smaller than the total dataset, and it is what you size a cache for.

Q2. How do you size a cache? BASIC

Estimate the working set, apply 80/20 to cache the hot fraction, multiply by entry size plus overhead, add ~25% headroom and replicas to get the node count, and check ops/s per node.

Q3. 100M unique links read per day, 800 B per entry. How big a cache? SCENARIO

20% of 100M = 20M entries × 800 B = 16 GB. A single 64 GB node holds it, plus a replica for availability.

Q4. You need 160 GB of cache on 64 GB nodes. How many nodes? SCENARIO

Use about 48 GB per node (25% headroom): 160 ÷ 48 = 3.3, round up to 4 primaries. One replica each: 8 nodes.

Q5. The hit rate drops from 95% to 70% over a month. Why, and what do you do? SCENARIO

The working set grew (more traffic, more unique keys) or eviction is wrong (no LRU, TTLs too long or too short). Check evictions and memory, add nodes or fix the policy.

Q6. Memory says 2 nodes are enough, but peak is 500,000 ops/s. How many? TRICKY

At ~100,000 ops/s per node, you need at least 5 primaries for throughput, more with headroom. Take the larger of the memory count and the throughput count.

Q7. Why not fill each Redis node to 100% of RAM? TRICKY

Spikes, fragmentation and snapshot forks need extra memory. At 100% the node evicts aggressively or runs out of memory. Keep about 25% free.

Next: SD-D-009 · How Many Servers Do You Need? Capacity, 60% Target & N+2 backendroadmap.com